# Step 9 Summary — Multi-Agent Reinforcement Learning

Research on the possibilities for applying Artificial Intelligence in computer games

Alexander Andreev

July 2026

## Contents



# 9 Step 9 — Multi-Agent Reinforcement Learning: Coordination, Competition, and Communication

This is a ground-up chapter on multi-agent reinforcement learning (MARL): the problem that makes it different from everything before it, the mathematics that frames it, the family of methods that attack it, and a set of controlled experiments run on small, exactly-solvable games. It serves two purposes — a self-contained refresher for later steps that build on it, and a primary source for the Part-1 thesis synthesis. It is written to be read on its own; no prior familiarity with the project's code is assumed. **All experimental numbers reported here were measured** on reproducible runs of the testbeds and, wherever possible, are bounded by *exact* analytical references (Nash equilibria, exact best-response values) rather than by other simulations. Where a run contradicted what theory led me to expect, I keep the original expectation and reconcile it with what happened — those gaps are the most instructive parts of the step.

**Where this sits in the thesis.** Steps 2–8 lived entirely inside **two-player zero-sum** games: there is a value $v^*$, a Nash strategy secures it against *any* opponent, and CFR provably converges to it. Step 9 is the pivot into the **multi-agent** world, where those three comforts weaken or vanish. It carries three thesis hooks. **LOLA** — differentiating through an opponent's *learning step* — is *dynamic* opponent modeling,

the moving-target complement to Step 7's static read (Contribution #1). **PSRO** — a game played over a *population of policies* — is both the framework for safe exploitation where there is no minimax theorem (Contribution #2) and a general multi-agent *evaluation methodology* (Contribution #3). The place where every two-player guarantee breaks — the missing minimax anchor for $N > 2$ — is named here and left open for the thesis to attack.

---

### 9.1 1. Why multi-agent RL is a different problem

In single-agent RL (Steps 1 and 6) an agent learns by trial and error against a **fixed** world. Formally it faces a stationary Markov decision process: the transition law $P(s' \mid s, a)$ and reward $R(s, a)$ do not change while the agent trains, which is exactly what lets a fixed optimal value function $Q^*$ exist for the agent to converge toward. In Steps 2–8 we went one level up and computed **equilibria** — strategies that are optimal against a *perfectly rational* opponent who has already finished reasoning.

Multi-agent RL sits between these two and is harder than both. Several agents **learn at the same time in a shared world**, so from any one agent's seat the "environment" — which now *includes the other agents* — **keeps changing** as everyone updates. This is **non-stationarity**, and it breaks the assumption underneath single-agent RL. From agent $i$'s perspective the effective transition and reward depend on the other agents' policies $\pi_{-i}$, and those are moving every update, so the target $Q^*$ that agent $i$ chases is itself in motion. It also creates two problems that simply do not exist for a lone agent: **coordination** (how do cooperating agents learn to act together without being told how?) and **credit assignment** (when the team succeeds, whose actions mattered?).

A picture to hold onto: **learning to dance with a partner who is also learning to dance.** If your partner's steps were fixed, you could memorize a routine that fits them — that is single-agent RL. If your partner were a flawless professional, you could study their known style and prepare the perfect counter — that is equilibrium computation. But when *both* of you are improving in real time, every adjustment you make changes what they should do, and vice-versa; you step on each other's toes, over-correct, and oscillate — until you either lock into a shared rhythm (a coordinated equilibrium) or cycle forever (as in Matching Pennies, §4). The entire field is the art of *stacking the deck so the synchronization happens reliably.*

> **Read more:** Zhang, K., Yang, Z. & Başar, T. (2021). "Multi-Agent Reinforcement Learning: A Selective Overview of Theories and Algorithms." *Handbook of RL and Control* — §1–2 for non-stationarity and the taxonomy used here.

---

### 9.2 2. Markov games — the bridge from Steps 2–8 to MARL

Steps 2–8 reasoned about **extensive-form games (EFGs)**: a game *tree*, **information sets** grouping histories a player cannot distinguish, and counterfactual values feeding a regret solver (CFR). Step 9

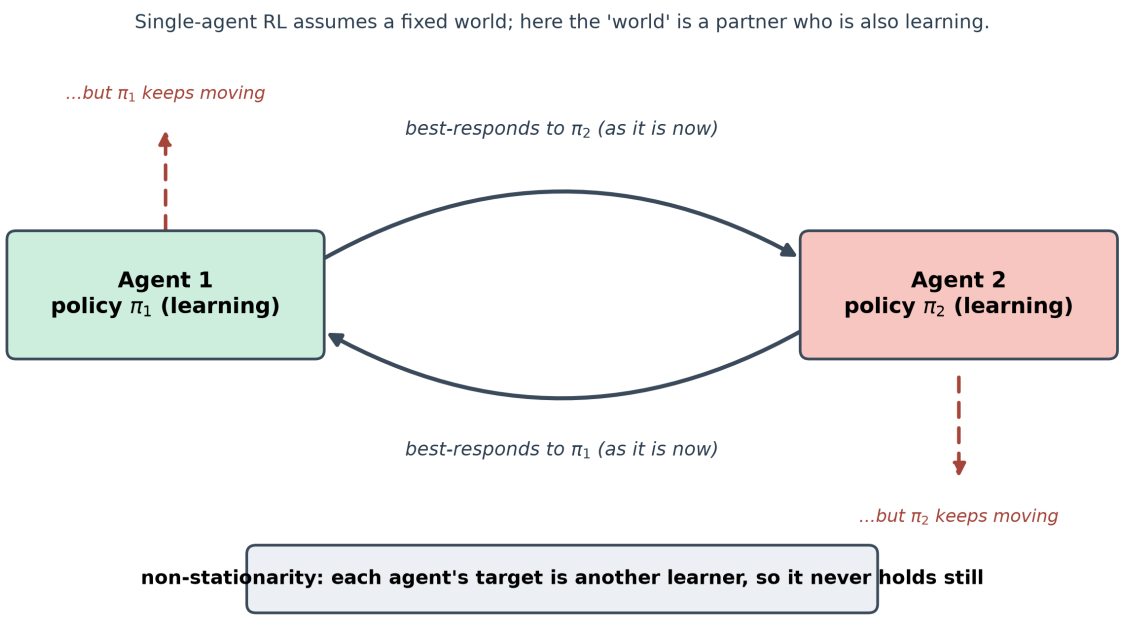


Figure 1: Non-stationarity as two partners learning to dance at once. Each agent optimizes against the other's current policy (the solid arrows), but that policy is itself moving because the other agent is optimizing back (the dashed arrows). The target each one chases is never still, so naive learners cycle or over-correct rather than converge. Every method in this chapter is a different way to make the moving target hold still long enough to learn against.

reasons about **Markov (stochastic) games**, the standard MARL formalism. The two connect cleanly, and stating the connection is what keeps the switch from feeling like starting over.

A **Markov game** is the tuple

$$(\mathcal{S},\ \{\mathcal{A}_i\}_{i=1}^{N},\ P,\ \{R_i\}_{i=1}^{N},\ \{\Omega_i\},\ O,\ \gamma),$$

with states $s \in \mathcal{S}$, one action set $\mathcal{A}_i$ per agent, a transition law $P(s' \mid s, a_1, \dots, a_N)$ driven by the **joint** action, per-agent rewards $R_i(s, a_1, \dots, a_N)$, and (in the partially observed case) observations $o_i = O_i(s)$. Each agent has a policy $\pi_i(a_i \mid o_i)$; together they form a **joint policy** $\pi = (\pi_1, \dots, \pi_N)$. The special cases are exactly the previous steps:

- $N = 1$ recovers an ordinary **MDP** (Step 1).
- $N = 2$, $R_1 = -R_2$, a single state gives a **matrix game** (this step's testbed, §4).
- Sequential, imperfect-information, zero-sum gives an **EFG** (Steps 2–8): a Markov game whose "state" is a *history* and whose partial observability is precisely an **information set**.

What survives the bridge is the *vocabulary*. A policy still maps what-you-know to a distribution over actions, so the EFG's behavioral strategy at an information set is exactly the Markov-game policy $\pi_i(a_i \mid o_i)$; the information set becomes the observation $o_i$; and the counterfactual value of a history becomes a centralized critic's estimate at a state. What does **not** survive is the *guarantees*. CFR's counterfactual

decomposition needs the tree and perfect recall, which general Markov games (loops, simultaneous moves, $N > 2$) do not provide, so "average strategy → Nash" no longer holds; MARL falls back to gradient/value learning whose convergence is not guaranteed (hence the cycling of §4). Most consequentially, the **minimax value anchor** disappears: in two-player zero-sum a Nash strategy guarantees $v^*$ against any opponent — the fact that made Step 8's safe exploitation coherent — and for $N > 2$ there is no such single value. That missing anchor is the precise gap Contribution #2 inherits.

> **Read more:** Littman, M. L. (1994). "Markov Games as a Framework for Multi-Agent Reinforcement Learning." *ICML* — the paper that introduced this framing and the minimax-Q algorithm.

---

### 9.3 3. The family of methods

Every method in this step is a different structural answer to "the other agents are learning while I learn." Five families matter here.

| Approach | What it does | Reach for it when | Main weakness |
|---|---|---|---|
| **Independent Learning (IL)** | each agent runs its own single-agent RL, treating others as environment | a quick baseline; near-stationary settings | non-stationarity → cycling, coordination failure |
| **MADDPG** (CTDE) | each agent's **critic** sees all agents' obs+actions at training; each **actor** sees only its own obs at execution | mixed cooperative-competitive tasks; the canonical CTDE template | critic input grows with $N$; the actor update is fiddly |
| **MAPPO** (CTDE) | plain PPO with a **centralized value** $V$(global state) and shared parameters | cooperative MARL where you want a simple, strong baseline | on-policy sample cost; "simple" but tuning-sensitive |
| **PSRO** | maintain a population of policies; solve a **meta-Nash** over it; train a **best response** to that mixture; add it; repeat | competitive/general games; when you want a game-theoretic convergence target | a full best response per round; an approximate oracle weakens the guarantee |

| Approach | What it does | Reach for it when | Main weakness |
|---|---|---|---|
| **LOLA** | optimize assuming the opponent takes **one learning step**; differentiate *through* their update | 2-player differentiable games where naive learning fails (IPD) | assumes you know and can differentiate the opponent's update |
| **CommNet** | agents broadcast a **differentiable message**; each receives the **mean** of the others' and feeds it into its policy | cooperative tasks with partial observability | mean pooling discards *who* said what |

Two axes cut across the table (Figure below). First, **what is centralized, and when?** IL centralizes nothing; CTDE centralizes the *critic/value at training only* (execution stays decentralized); PSRO centralizes a whole *meta-game solve* between training rounds; CommNet centralizes *information at execution* through a learned channel. Second, **is the opponent modeled as static or as a learner?** Everything except LOLA treats the opponent's strategy as fixed-while-you-respond; **LOLA** anticipates the opponent's *next update* — dynamic, not static.

Historically the line runs: learn to talk (CommNet, 2016) → centralize the critic (MADDPG, 2017) → bring game theory to a population (PSRO, 2017) → factorize the value (QMIX, 2018) → look ahead at the opponent's learning (LOLA, 2018) → and then discover that the simple thing often wins (MAPPO, 2022).

> **Read more:** Albrecht, S. V., Christianos, F. & Schäfer, L. (2024). *Multi-Agent Reinforcement Learning: Foundations and Modern Approaches* (MIT Press), Ch. 8–9 — a current textbook treatment of exactly this taxonomy.

---

### 9.4 4. Independent learning and its failure modes

The cleanest way to see *why* the rest of the field exists is to run the control that fails. Independent learning drops each agent into its own gradient loop and pretends the others are part of the floor. On four canonical $2 \times 2$ matrix games — each a different qualitative case — it produces four different fates.

The games and their analytic Nash equilibria (ground truth): **Prisoner's Dilemma** (a dominant strategy: Defect strictly beats Cooperate, so the unique Nash is mutual Defect); **Matching Pennies** (zero-sum; the unique Nash is the fully mixed $(\frac{1}{2}, \frac{1}{2})$; value 0); **Stag Hunt** (two pure Nash — the payoff-dominant (Stag, Stag) and the risk-dominant (Hare, Hare) — plus a mixed one); **Battle of the Sexes** (two pure Nash that disagree on which to pick, plus a mixed one).

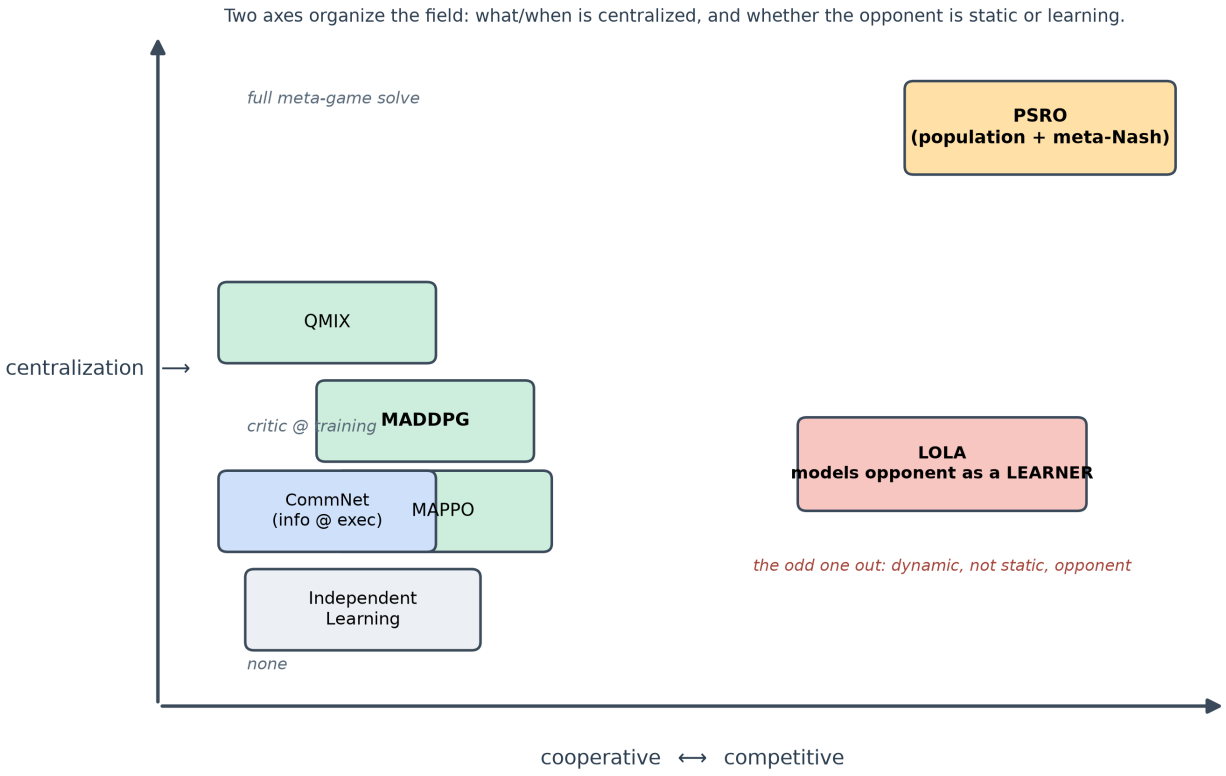


Figure 2: The method family on two axes. Horizontal: how competitive vs cooperative the target setting is (IL and PSRO span competition; MADDPG/MAPPO/QMIX/CommNet target cooperation; LOLA is the mixed-motive bridge). Vertical: how much is centralized and when (nothing for IL; the critic at training for CTDE; a full meta-game solve for PSRO; information at execution for CommNet). LOLA is the outlier that models the opponent as a learner rather than a fixed strategy.

Running two independent gradient learners (exact gradients, so the dynamics are clean and noise-free) gives, measured:

| Game | Measured outcome | NashConv | Matches analytic Nash? |
|---|---|---|---|
| Prisoner's Dilemma | $x \to [0.001, 0.999]$ (Defect) | 0.0013 | yes — the dominant-strategy equilibrium |
| Stag Hunt | all seeds → (Hare, Hare) | 0.0013 | yes — the *risk-dominant* pure Nash |
| Battle of the Sexes | all seeds → one pure Nash | 0.0013 | yes — a pure Nash |
| Matching Pennies | **does not converge** (last iterate drifts to the boundary) | 1.4–1.8 | no — it never settles |

Three of the four converge to a genuine Nash, and the interesting details are in the "how." In Stag Hunt the learners reliably pick the **risk-dominant** equilibrium (Hare) rather than the payoff-dominant one (Stag) even though Stag pays more — a unilateral move toward Stag is punished, so gradient dynamics slide to the safe corner. In Battle of the Sexes every seed lands on the *same* pure equilibrium; the learners solve the coordination problem but not in a seed-diverse way. These are the coordination and equilibrium-selection problems made concrete.

Matching Pennies is the headline: it **never converges**, which is exactly the point.

> **Reconciliation (kept prediction → what actually happened).** I predicted Matching Pennies would trace a clean orbit at roughly constant radius around $(\frac{1}{2}, \frac{1}{2})$. Two learners, run two ways, both failed to converge — but neither orbited cleanly. The projected-gradient (IGA) learner used in the exploration scripts *spirals outward*: the measured distance-to-Nash grew across time-windows from 0.30 to 0.48. The softmax-logit learner used in the implementation *drifts to the corners* (final profiles like $x = [0.96, 0.04]$, $y = [0.03, 0.97]$, NashConv $\approx 1.8$). The **lesson is unchanged and arguably sharper**: under naive simultaneous learning the *last iterate* does not converge in a game with only a mixed equilibrium — the thing that converges is the *time-average* (§6). The specific "energy-preserving orbit" mental model was the part that was wrong; the mechanism is a slow divergence toward the boundary, which is if anything a stronger argument for the population/averaging machinery that follows.
>
> **Read more:** Singh, S., Kearns, M. & Mansour, Y. (2000). "Nash Convergence of Gradient Dynamics in General-Sum Games." *UAI* — the analysis of why gradient ascent cycles rather than converges in games like Matching Pennies.

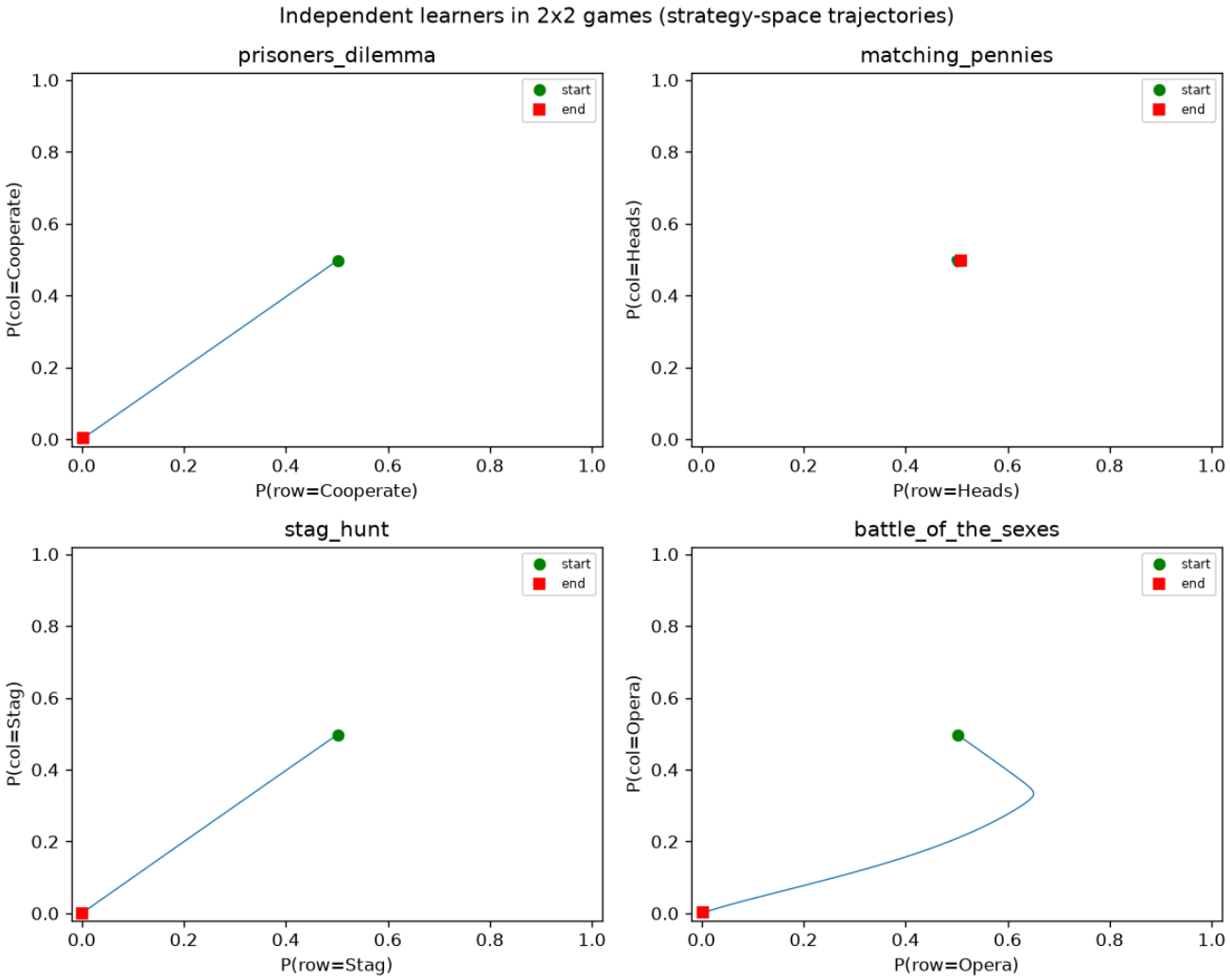


Figure 3: Independent learners on the four matrix games: Prisoner's Dilemma collapses to mutual defection, Stag Hunt and Battle of the Sexes settle on a pure equilibrium, and Matching Pennies fails to converge (its trajectory drifts away from the mixed Nash rather than settling on it). The contrast is the visceral case for coordination machinery.

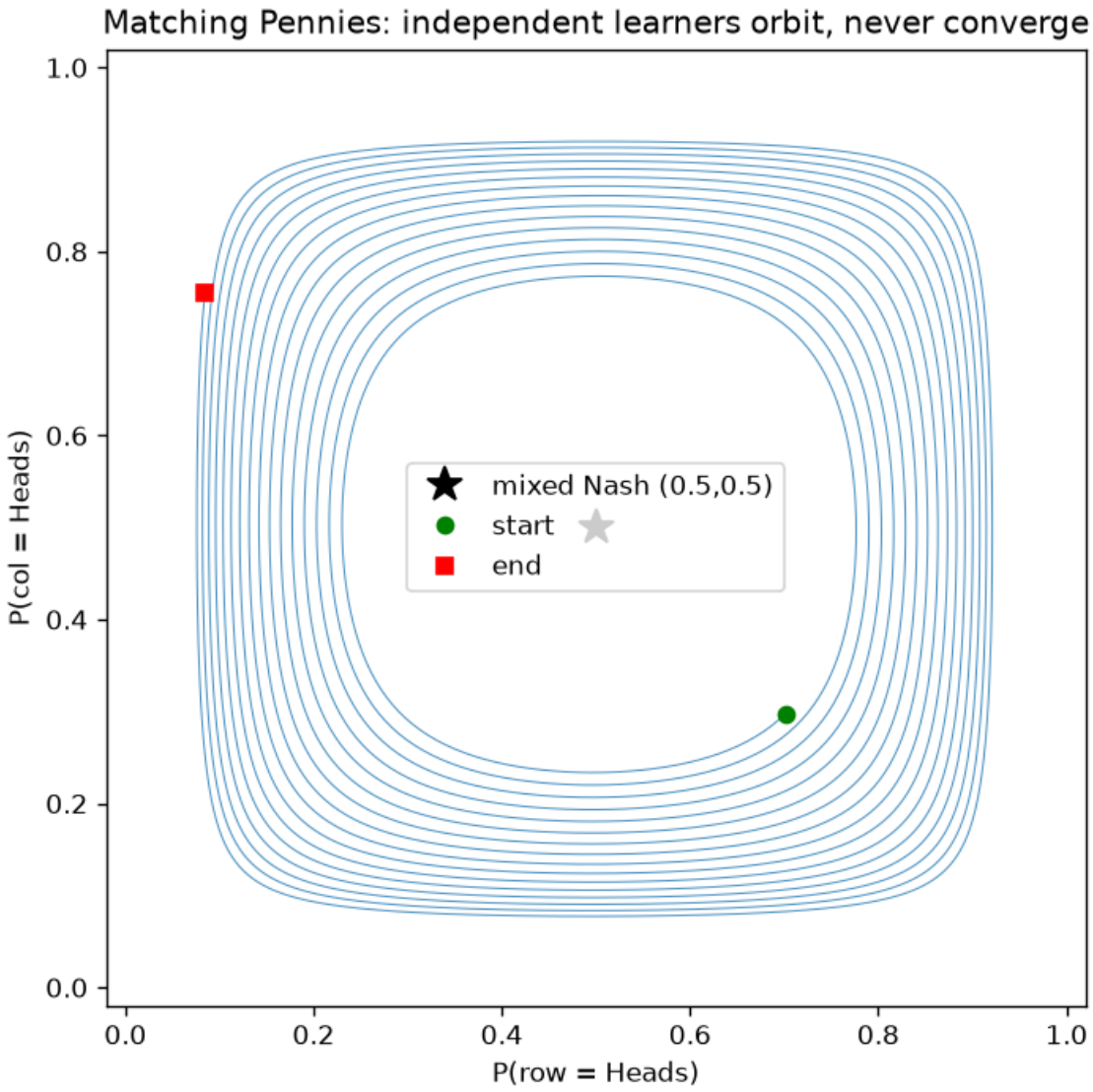


Figure 4: Zoom on non-stationarity: the distance-to-Nash for Matching Pennies does not shrink (it grows across time-windows, 0.30 to 0.48), while the Prisoner's Dilemma distance-to-(Defect,Defect) collapses to zero. More compute traces a bigger divergence, not convergence — non-stationarity is structural, not a budget problem.

## 9.5 5. Centralized training, decentralized execution (CTDE)

The first structural fix keeps execution realistic — each agent still acts on its own observation — while giving the *learner* a privileged view during training. The archetype is **MADDPG** (Lowe et al., 2017): each agent's **actor** $\pi_i(a_i \mid o_i)$ sees only its own observation, but a **centralized critic** $Q_i(s, a_1, \ldots, a_N)$ sees the global state and *every* agent's action. The asymmetry is the whole idea. A per-agent critic that watches only $o_i$ faces a non-stationary, partially-observed world, so identical inputs map to different returns and its value target is noisy; a centralized critic conditioned on everything sees a target that is (near-)deterministic, so it is a far lower-variance teacher. **MAPPO** (Yu et al., 2022) is the minimal version: plain PPO with a single **centralized value function** $V$(global state) instead of per-agent $V(o_i)$, and it is often the strongest baseline — a caution against over-engineering.

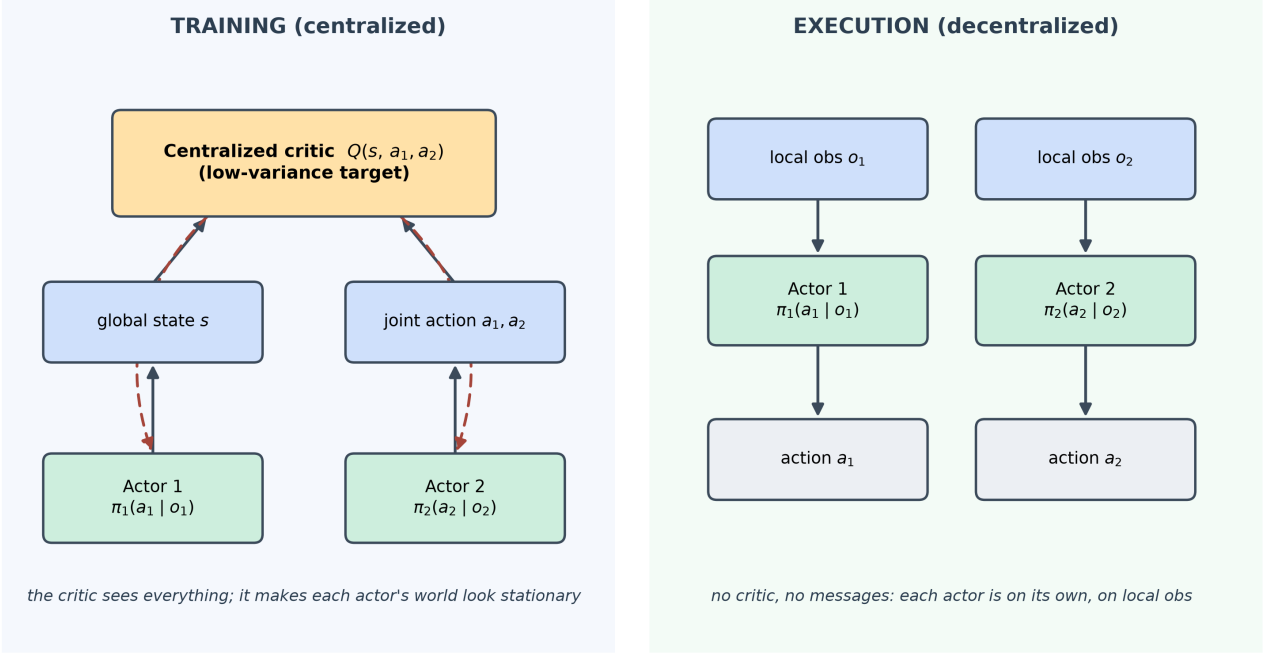


Figure 5: CTDE architecture. During training a centralized critic (or value function) sees the global state and the joint action and produces low-variance value targets; during execution each actor acts on its own local observation with no access to the critic and no message passing. The training/execution asymmetry is what makes the world look stationary to the learner without cheating at showtime.

**Does the centralized critic actually have lower variance?** Measured on a one-step cooperative "referential" task (a speaker sees a target, a listener does not, both must name it), and trained to convergence:

| Critic | Final residual (value loss) |
|---|---|
| centralized $Q(s, \text{joint } a)$ | $3.2 \times 10^{-11}$ |
| independent $Q_i(o_i)$ | 0.077 |

The centralized critic drives its residual to essentially zero — it can see the target and the joint action, so the reward is a deterministic function of its inputs — while the independent critic cannot see the target and is stuck predicting the base rate. This is the CTDE variance-reduction claim, confirmed cleanly.

But variance reduction is **not** the same as solving coordination, and this is where a prediction broke.

> **Reconciliation (kept prediction → what actually happened).** I predicted that on the Claus–Boutilier **climbing game** — a stateless cooperative matrix game whose optimum (11) is flanked by −30 miscoordination penalties, with a "safe" attractor at 5 — the centralized critic would escape the trap and reach the optimum, beating independent learners. It did not. Measured greedy rewards: **independent learners 7, MADDPG 5, MAPPO 7** (optimum 11, safe 5). No method reached the optimum, and MADDPG actually *underperformed* both IL and MAPPO. The honest reading: a centralized critic lowers value-target variance (confirmed above) but that is **not sufficient** to overcome relative over-generalization plus the hard-exploration risk of the −30 penalties — the agents will not try the risky joint action long enough to discover the 11. MADDPG's below-IL result specifically flags its discrete counterfactual-baseline actor update as the piece to scrutinize next. The lesson survives in a chastened form: **CTDE buys a better critic, not automatic coordination.**

A methodological note worth carrying forward: both CTDE effects above were **invisible at the fast "smoke" configuration** — there the critic losses were near-equal and the communication benefit (§7) was zero. They appeared only once trained at the larger "scale" configuration. The smoke config proves the code runs; the phenomena need training to convergence.

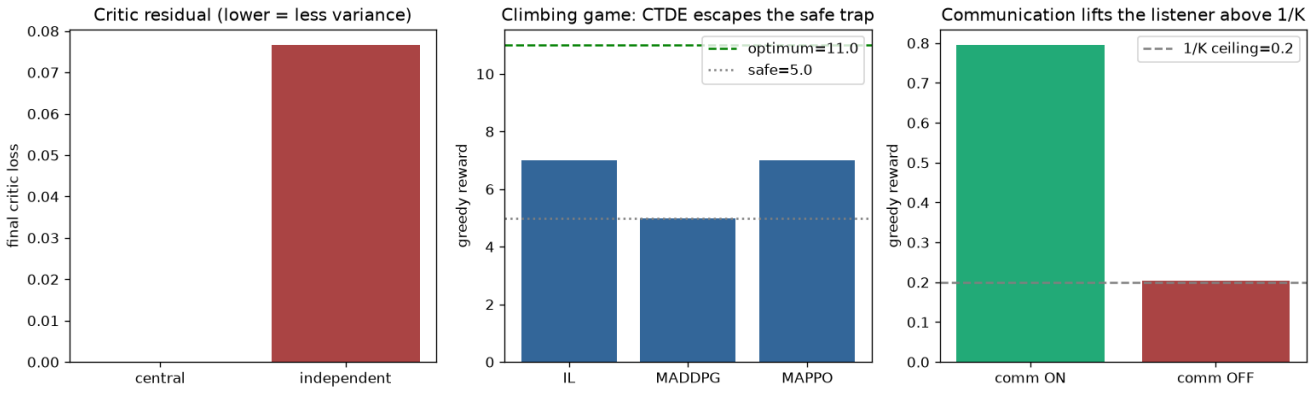


Figure 6: Cooperative CTDE and communication results (scale config). Left: the centralized critic's residual is orders of magnitude below the independent critic's. Middle: on the climbing game, no method reaches the optimum (11); MADDPG (5) trails IL and MAPPO (7) — CTDE reduces critic variance but does not by itself solve hard-exploration coordination. Right: communication lifts the listener far above the 1/K guessing ceiling (see §7).

> **Read more:** Lowe, R. et al. (2017). "Multi-Agent Actor-Critic for Mixed Cooperative-Competitive Environments." *NeurIPS* (MADDPG); and Yu, C. et al. (2022). "The Surprising Effectiveness of PPO in Cooperative Multi-Agent Games." *NeurIPS* (MAPPO).

### 9.6 6. PSRO — game theory over a population of policies

The second structural fix is the through-line of the step and the direct descendant of Step 2's iterated best response. **PSRO** (Policy-Space Response Oracles; Lanctot et al., 2017) treats whole policies as the atoms of a higher game. It maintains a **population** of policies per player, builds the empirical **meta-game** payoff matrix between the populations, solves a **meta-Nash** over it, trains a **best response** (the "oracle") to the opponent's meta-Nash mixture, and adds that response to the population — repeat. It unifies self-play, fictitious play, and the double-oracle method under one framework, and, crucially for this project, its progress metric is *exploitability* — the very same NashConv used throughout Steps 2–8.

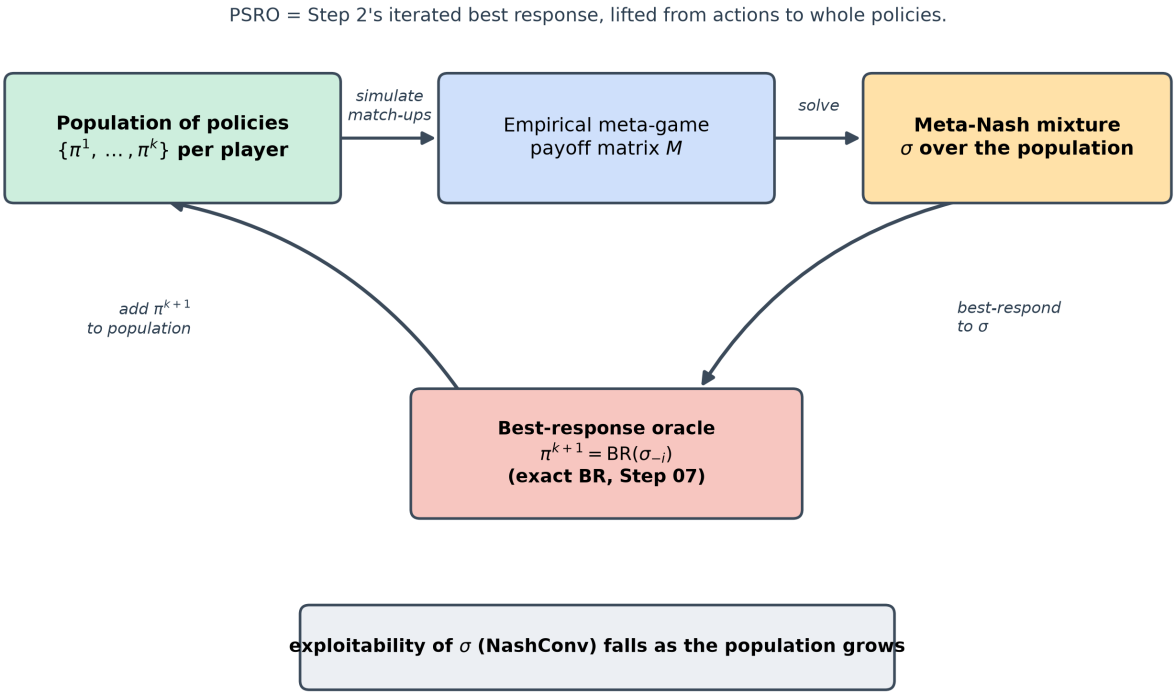


Figure 7: The PSRO double-oracle loop. From the current populations, build the meta-game payoff matrix, solve its meta-Nash mixture, then call a best-response oracle against the opponent's mixture and add the new policy to the population. Exploitability of the meta-Nash mixture is expected to fall as the population grows. In this project the oracle is Step 07's exact best response, so PSRO's convergence is measured with the same exploitability yardstick as the game-theory steps.

Two facts make the implementation exact rather than approximate. First, the oracle reuses Step 07's **exact best response** on Kuhn and Leduc. Second, the opponent's meta-Nash mixture over *behavioral policies* is, by Kuhn's theorem (these games have perfect recall), realization- equivalent to a **single behavioral policy**; collapsing the mixture that way lets the exact best-response engine apply directly.

**Why a population and not just the last self-play policy?** Because self-play converges in the *average*, not the last iterate. Measured on Kuhn with fictitious-play self-play: the **average**-iterate exploitability

fell from 0.24 to 0.031 over 200 iterations, while the **last**-iterate exploitability kept oscillating between 0.33 and 0.83. Trusting the latest policy would be trusting a number that never settles; PSRO's meta-mixture is the population-level version of the averaging that CFR and fictitious play rely on.

**PSRO convergence, measured** (exploitability = NashConv of the meta-Nash mixture in the full game):

| Game | Exploitability trajectory | Verdict |
|---|---|---|
| Kuhn Poker | $0.917 \to \sim 2 \times 10^{-16}$ by round 6 | converges to machine zero |
| matrix (Matching Pennies) | $2.0 \to 0$ by round 2 | converges |
| Rock–Paper–Scissors | $2.0 \to 0.017$; population $\to$ | converges |
| (exploration) | {R,P,S}, mixture $\to$ uniform | |
| Leduc Hold'em | $4.75 \to 2.16$ over 20 rounds | decreases, but far above target |
| Goofspiel ($K = 3$) | $1.33 \to 0$ | converges |
| Goofspiel ($K = 4$) | oscillates $1.4 \leftrightarrow 2.0$ | does not settle |

The Kuhn / matrix / RPS results are textbook: on the small games the population quickly spans the strategies needed and exploitability collapses. Two results did not go as predicted.

> **Reconciliation 1 (Leduc).** I predicted PSRO would drive Leduc exploitability below 0.5 within 20 iterations. Measured, it fell from 4.75 to 2.16 — a clear, roughly monotone decline, but nowhere near 0.5. This is **genuine slow convergence**, not a bug: Kuhn hit machine zero in 6 rounds, but Leduc's game tree is far larger, and a population of 20 *pure* best responses is simply too small to closely approximate its mixed Nash. The "$< 0.5$ in 20" target was optimistic. The lesson — exploitability decreases as the population grows — holds; the *rate* is the scaling wall, and it rhymes with Step 8's global-vs-local scaling finding.
>
> **Reconciliation 2 (Goofspiel $K = 4$).** I predicted non-increasing exploitability. At $K = 3$ it converged to 0 cleanly; at $K = 4$ it oscillates between $\sim$1.4 and $\sim$2.0 and does not settle. This is the one result I cannot yet fully explain, and per the workflow I am **documenting it, not fixing it**. Two concrete suspects for a follow-up session: the Goofspiel PSRO driver never de-duplicates best-response policies (so the meta-game can stall on repeats), and a pure-strategy population is likely too weak to represent the larger game's mixed meta-Nash. Flagged as an open code item, not a validated result.
>
> **Read more:** Lanctot, M. et al. (2017). "A Unified Game-Theoretic Approach to Multiagent Reinforcement Learning." *NeurIPS* (PSRO); and McMahan, H. B., Gordon, G. & Blum, A. (2003). "Planning in the Presence of Cost Functions Controlled by an Adversary." *ICML* (the double-oracle method PSRO generalizes).

---

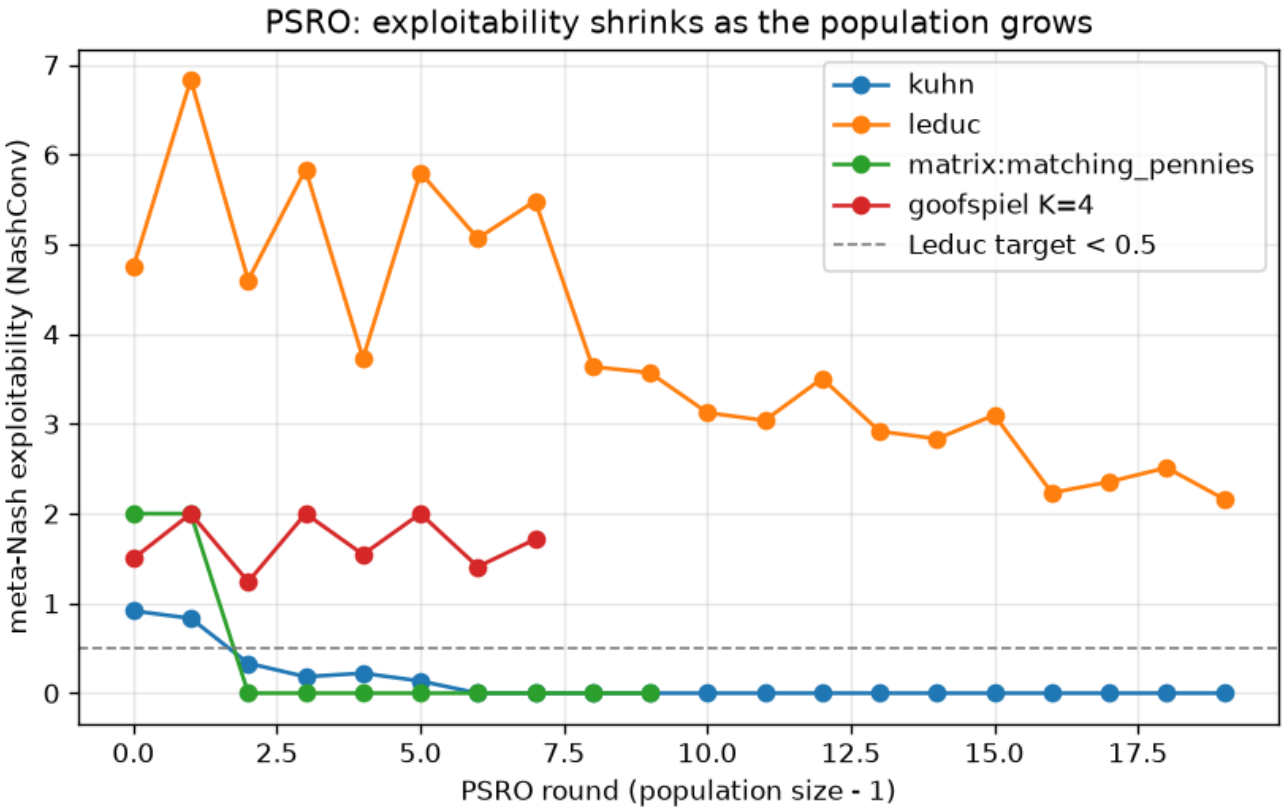


Figure 8: PSRO exploitability vs population size across games (scale config). Kuhn, the matrix game, and Rock-Paper-Scissors collapse to (near) zero within a handful of rounds; Leduc declines steadily but stays well above the 0.5 target after 20 rounds (a scaling wall for a pure-strategy population); Goofspiel K=4 oscillates rather than settling (a flagged anomaly). The exact best-response oracle guarantees convergence in principle; the rate is what size controls.

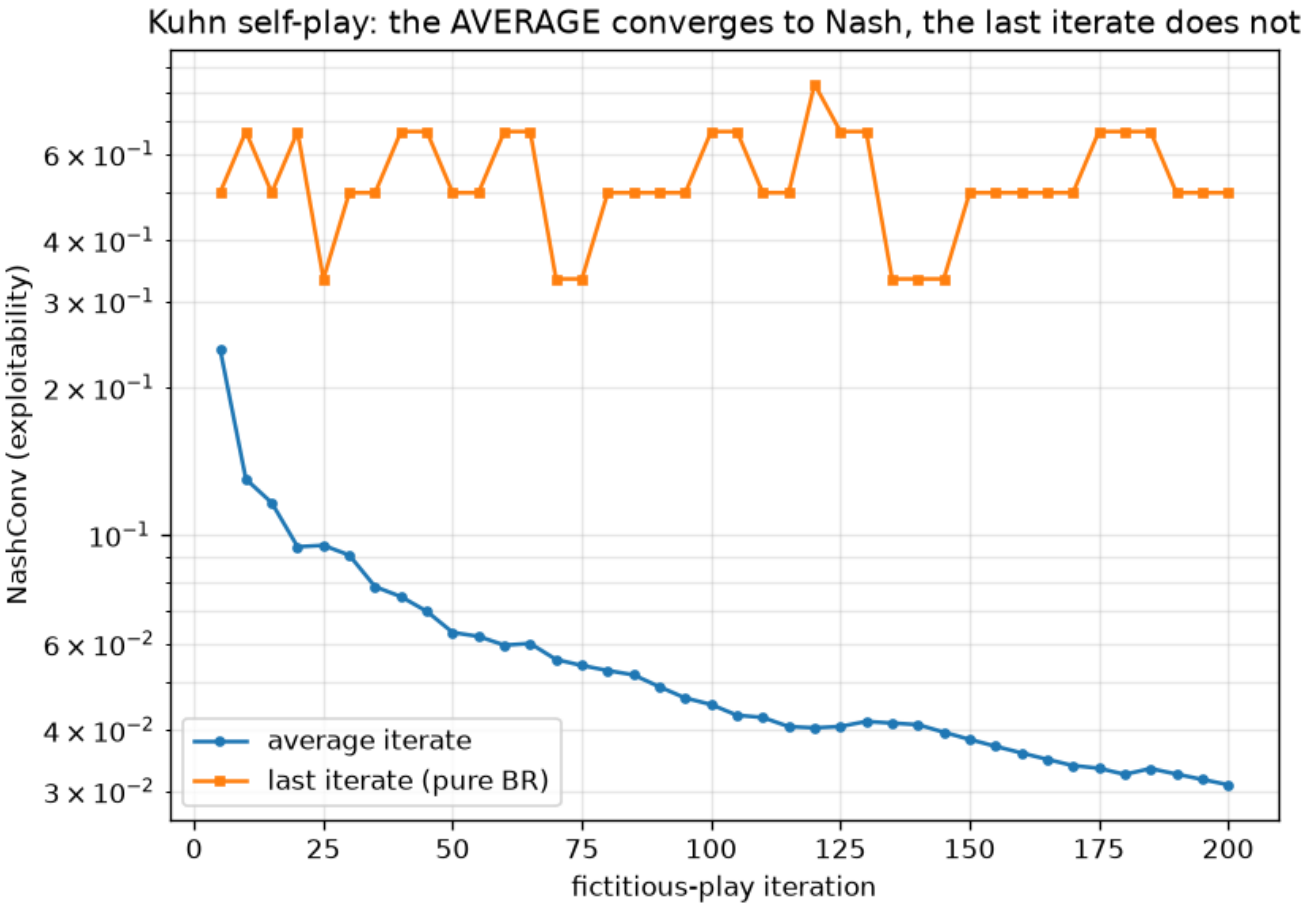


Figure 9: Self-play on Kuhn: the average-iterate exploitability (NashConv) falls steadily toward zero while the last-iterate exploitability keeps oscillating. This is why self-play and PSRO rely on averaging over a population rather than trusting the most recent policy.

### 9.7 7. Learned communication (CommNet)

CTDE centralizes information at *training*; communication centralizes it at *execution*, through a channel the agents **learn** rather than one a designer builds. In **CommNet** (Sukhbaatar et al., 2016) each agent encodes its observation to a hidden vector, emits a **message**, and then receives the **mean** of the other agents' messages as extra input to its policy. The whole thing is trained end-to-end, so the protocol emerges: on the referential task, the speaker (who alone sees the target) must learn to encode it and the listener must learn to decode it.

The test is sharp because without a channel the listener is capped at pure guessing, $1/K$. Measured (scale config, $K = 5$, so the guessing ceiling is 0.2):

| Channel | Greedy team reward |
|---|---|
| communication ON | 0.795 |
| communication OFF | 0.204 |

With the channel the listener climbs well above the $1/K$ ceiling; without it, it sits exactly at the ceiling. Communication is doing real work — and note the reconciliation from §5 applies here too: at the smoke configuration both numbers were 0.24 (the channel had not yet learned to carry information), and the benefit appeared only after training at scale.

> **Read more:** Sukhbaatar, S., Szlam, A. & Fergus, R. (2016). "Learning Multiagent Communication with Backpropagation." *NeurIPS* (CommNet).

---

### 9.8 8. LOLA — modeling the opponent as a learner

Every method so far treats the opponent's strategy as fixed while you respond. **LOLA** (Learning with Opponent-Learning Awareness; Foerster et al., 2018) is the exception, and it is the one most directly connected to the thesis. Instead of optimizing against the opponent's *current* policy, each agent optimizes against the policy the opponent will hold *after one learning step*, and differentiates *through* that step. The extra term is a mixed second derivative — how the opponent's update depends on *my* parameters — and it is what turns self-interested agents cooperative.

The classic demonstration is the memory-1 **Iterated Prisoner's Dilemma**, where each agent's policy is five cooperation probabilities and the expected discounted return has a closed form via the stationary Markov chain over outcome pairs. Naive gradient learners, each maximizing its own return against the other's current policy, converge to **mutual defection**; LOLA learners, each accounting for the other's upcoming update, reach **mutual cooperation**.

Measured (per-step discounted return; full cooperation $\approx 3$, mutual defection $\approx 1$):

| Learners | Return |
|---|---|
| naive vs naive | 1.04 |
| LOLA vs LOLA | 2.82 |

The direction is exactly the LOLA result — cooperation emerges where naive learning defects. (My prediction of $\approx 3$ was slightly high; the measured 2.82 is near-cooperation, and the exploration run with a larger look-ahead reached $\sim$2.9.) A built-in sanity check confirms the mechanism: setting the look-ahead learning rate to zero makes LOLA's gradient reduce exactly to the naive gradient, so the cooperation comes specifically from the second-order look-ahead term.

Conceptually this is **dynamic** opponent modeling: Step 7 inferred an opponent's *current* strategy; LOLA anticipates their *learning trajectory*. Combining the two — a static read that seeds a dynamic look-ahead — is a candidate for Contribution #1, not a solved thing.

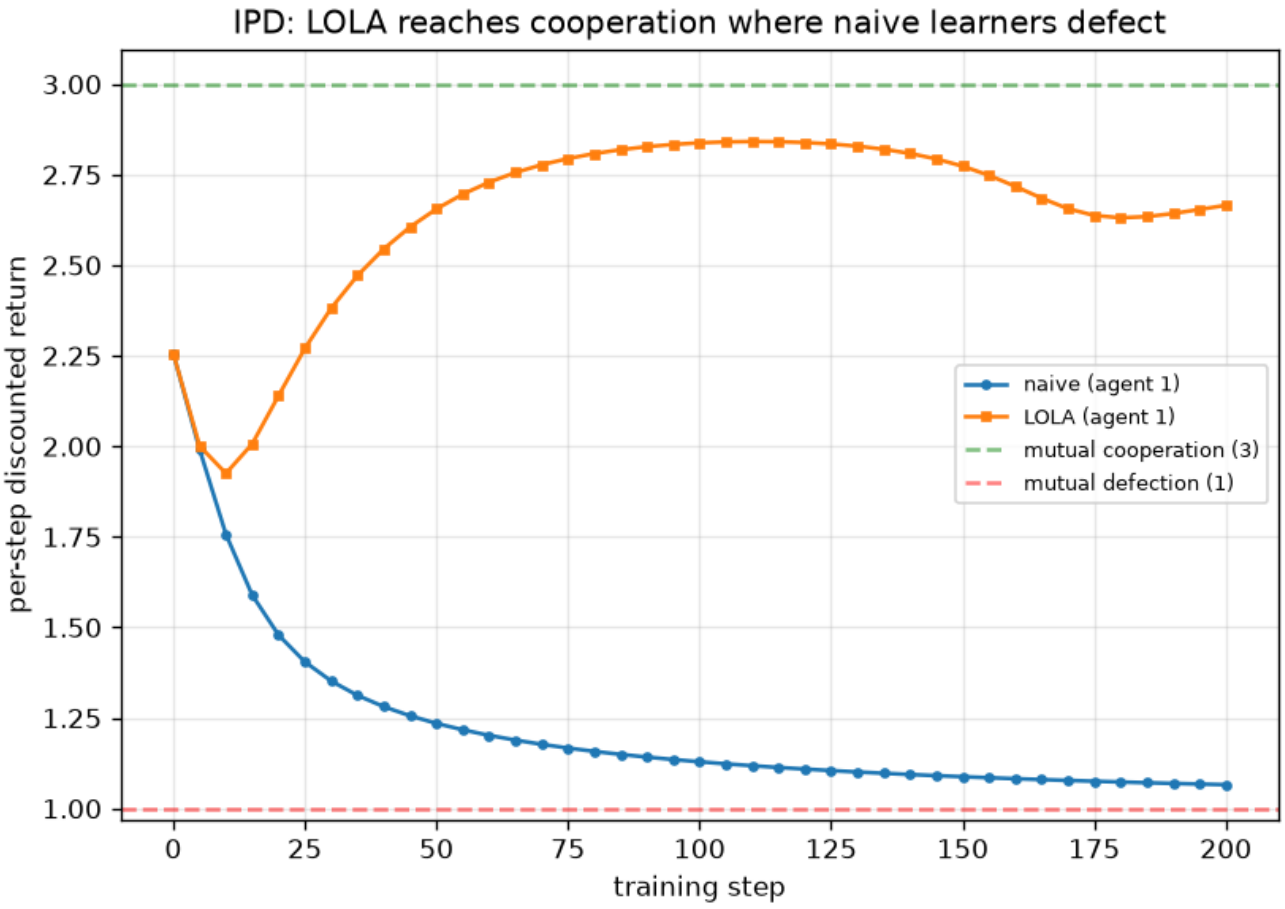


Figure 10: LOLA vs naive learners on the Iterated Prisoner's Dilemma: naive learners' per-step return collapses toward mutual defection (~1), while LOLA learners' return climbs toward mutual cooperation (~2.8). Anticipating the opponent's next learning step is what reshapes the dynamics from defection to cooperation.

**Read more:** Foerster, J. et al. (2018). "Learning with Opponent-Learning Awareness." *AAMAS* (LOLA).

---

### 9.9 9. Honest notes, limitations, and where this hands off

**What held up.** The confirmed results are the backbone: independent learning converges on dominant/coordination games and *fails* on Matching Pennies; PSRO drives exploitability to (near) zero on Kuhn, the matrix game, and RPS with an exact oracle; self-play converges in the average but not the last iterate; the centralized critic has orders-of-magnitude lower residual variance; learned communication lifts the listener above the guessing ceiling; and LOLA turns IPD defectors into cooperators. Taken together they trace the intended arc from "why naive learning breaks" to "the structural fixes that repair it."

**What did not, and why it matters.** Four honest caveats travel forward. (1) Matching Pennies diverges toward the boundary rather than orbiting — a sharper version of the non-convergence lesson. (2) PSRO on Leduc converges *slowly* (exploitability $\sim$2.16 after 20 rounds, not $< 0.5$) — the pure-strategy-population scaling wall. (3) Goofspiel $K = 4$ oscillates rather than converging — a flagged, unexplained code anomaly (documented, not fixed). (4) On the climbing game no method reached the optimum and MADDPG underperformed independent learners — a centralized critic reduces variance but does not by itself solve hard-exploration coordination. The two code items (Goofspiel $K = 4$; MADDPG's counterfactual baseline) should be investigated before those pieces are reused. And a methodological point: the neural effects were invisible at the fast smoke config and only emerged at scale, so the scale numbers are the ones the claims rest on.

**Trust.** Every equilibrium target is *exact* (analytic Nash for the matrix games; Step 07's exact best response and NashConv for Kuhn/Leduc/Goofspiel), so the game-theoretic results are bounded by ground truth rather than by other simulations. The neural results are qualitative inequalities (central $<$ independent; comm ON $>$ comm OFF), not precise values, and are seed- and version-sensitive by construction. The experiment PNGs cited above are generated from the committed JSON artifacts (see `../figures/README.md`); a few were not saved on the run and are marked "to close."

**Backward and forward connections.** Backward: PSRO is Step 2's iterated best response lifted to a population, and it reuses Step 07's exact best-response engine wholesale; the Leduc scaling wall echoes Step 8's global-vs-local finding. Forward: the three thesis hooks are now concrete — LOLA as dynamic opponent modeling (Contribution #1), PSRO's meta-game as an evaluation methodology (Contribution #3), and, above all, the **missing $N > 2$ minimax anchor** (Contribution #2), the single place where every two-player guarantee from Steps 2–8 stops applying.

**Open questions.** Can an approximate (RL) oracle push Leduc PSRO much closer to Nash, and at what cost to the convergence guarantee? What breaks the Goofspiel $K = 4$ oscillation — de- duplication, a mixed-strategy population, or a different meta-solver? Why does a centralized critic *hurt* on the climbing game, and does a counterfactual/COMA baseline fix it? And the thesis question underneath all of them: with no minimax value for $N > 2$, what does "safe" even mean for a population, and can PSRO's

meta-game supply a usable substitute?

---

### 9.10 Key takeaways for the thesis synthesis

- **Non-stationarity is *the* problem**, and it is structural, not a compute-budget issue — Matching Pennies diverges no matter how long you train.
- **CTDE centralizes the critic at training and decentralizes the actor at execution**; measured, it delivers a near-zero-variance critic ($3.2 \times 10^{-11}$ vs 0.077) — but variance reduction alone did not solve the climbing game, so it is necessary, not sufficient.
- **PSRO is the game-theory MARL bridge**: measured, it drives Kuhn to machine-zero exploitability and RPS to uniform; Leduc declines but hits a scaling wall — the same wall that motivates the thesis's scalable methods.
- **Self-play/PSRO succeed in the average/population, not the last iterate** (Kuhn average NashConv $0.24 \to 0.031$ while the last iterate oscillates) — the reason a population exists.
- **LOLA reframes the opponent as a learner** and turns IPD defection into cooperation ($1.04 \to 2.82$) — the dynamic complement to Step 7's static opponent model (Contribution #1).
- **The $N > 2$ minimax gap is the open door to the thesis** (Contribution #2): the vocabulary of Steps 2–8 crosses into MARL, but the safety anchor does not.